



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AND SIMULATION OF AN ARITHMETIC LOGIC UNIT FOR PROCESSOR OPERATIONS

A CAPSTONE PROJECT REPORT

CSA1218 – COMPUTER ARCHITECTURE

Submitted by
SANTHOSH KUMAR M (192511017)

Under the Guidance of
DR. JAYASAKTHI VELMURUGAN K

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Design and Simulation of an Arithmetic Logic Unit for Processor Operations” has been carried out by Santhosh Kumar M under the supervision of Dr. Jayasakthi Velmurugan K and is submitted in partial fulfilment of the requirements for the current semester of the Computer Science and Engineering program at SIMATS Engineering, Chennai.

COURSE COORDINATOR	PROJECT GUIDE
Dr.V.Sarala Professor Department of Computer Science Engineering Saveetha School of Engineering, Saveetha Institute of Medical and Technical Sciences, SIMATS, Chennai	Dr.K.Jayasakthi Velmurugan Professor Department of Computer Science Engineering Saveetha School of Engineering, Saveetha Institute of Medical and Technical Sciences, SIMATS, Chennai

DECLARATION

I, Santhosh Kumar M of the Computer Science and Engineering Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design and Simulation of an Arithmetic Logic Unit for Processor Operations” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original and has been carried out in accordance with engineering ethics and academic integrity. All sources and software resources used in the project have been appropriately acknowledged.

Place: Chennai

Date: September 2026

Signature of the Students

INDIVIDUAL CONTRIBUTION STATEMENT

Student / Reg. No.	Responsibilities	Design & Development	Testing & Analysis	Report Contribution	%
Hemanth R / 192521327	Processor architecture & ALU	ALU logic/integration	Functional verification	Ch. 1–3	50%
Santhosh Kumar M / 192511017	Testing & documentation	Result/flag handling	Performance comparison	Ch. 4–5	50%
Total					100%

ABSTRACT

The Arithmetic Logic Unit (ALU) is a core datapath component of a processor and performs the arithmetic and logical operations required during instruction execution. This project presents the design and simulation of an 8-bit ALU integrated into a simplified RISC-style processor environment. The system is intended to make processor execution easier to understand through an interactive web-based simulator.

The ALU supports ADD, SUB, MUL and DIV arithmetic operations and AND, OR and XOR logical operations. The processor model includes Instruction Memory, Instruction Decoder, Control Unit, Register File, ALU, Flags and Result Register. Registers and operand values are automatically assigned, while the user selects the required operation. The system generates results and status flags and records execution information for graphical analysis.

Python is used for the simulation logic, Flask for the web application, HTML5/CSS3 for the interface, JavaScript for interaction and Chart.js for graphical representation. Testing verifies the supported operations and representative result behaviour. The system provides a compact educational model for understanding the relationship between instruction decoding, control, registers, ALU computation and status information.

Keywords: Arithmetic Logic Unit, 8-bit Processor, RISC, Processor Simulation, Instruction Execution, Computer Architecture

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	1
2	Declaration	2
3	Individual Contribution	3
4	Abstract	4
5	List of Figures	5
6	List of Tables	6
7	List of Abbreviations	7
8	CHAPTER 1 – INTRODUCTION AND ENGINEERING PROBLEM	8
8.1	1.1 Background	8
8.2	1.2 Problem Statement	9
8.3	1.3 Objectives	9
8.4	1.4 Scope of the Project	10
8.5	1.5 Functional Requirements	10
9	CHAPTER 2 – LITERATURE REVIEW AND EXISTING SOLUTIONS	11
9.1	2.1 Literature Review	11
9.2	2.2 Existing ALU Approaches	12
9.3	2.3 Limitations of Existing Approaches	13
10	CHAPTER 3 – ENGINEERING DESIGN AND TRADE-OFFS	14
10.1	3.1 Proposed ALU Architecture	14
10.2	3.2 ALU Functional Units	15
10.3	3.3 Instruction and Control Design	16

S. No.	Contents	Page No.
10.4	3.4 Design Alternatives and Trade-offs	17
10.5	3.5 Sustainability, Ethics and Safety	18
10.6	3.6 Risk and Security Considerations	19
11	CHAPTER 4 – IMPLEMENTATION, TESTING AND RESULTS	20
11.1	4.1 Implementation Methodology	21
11.2	4.2 Simulation Procedure	21
11.3	4.3 Test Plan	22
11.4	4.4 Test Results	23
11.5	4.5 Project Management	24
12	CHAPTER 5 – CONCLUSION, FUTURE WORK AND REFLECTION	25
12.1	5.1 Conclusion	25
12.2	5.2 Future Enhancements	26
12.3	5.3 Learning Outcomes and Reflection	26
13	References	27
14	Appendices	28

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Figure 1	Proposed 8-bit RISC Processor / ALU Architecture	14
Figure 2	ALU Instruction Execution Flow	16
Figure 3	Functional Blocks of the 8-bit ALU	15
Figure 4	Web-Based ALU Simulation Interface	21

LIST OF TABLES

Table No.	Title of Table	Page No.
Table 1	Project Requirements	10
Table 2	ALU Design Specifications	15
Table 3	Design Alternatives and Trade-offs	17
Table 4	Risk Assessment	19
Table 5	Test Plan	22
Table 6	Representative Test Results	23
Table 7	Project Objectives and Outcomes	24
Table 8	Team Roles and Responsibilities	24
Table 9	Estimated Project Budget	25

LIST OF ABBREVIATIONS / SYMBOLS

Term	Description	Unit
ALU	Arithmetic Logic Unit	—

RISC	Reduced Instruction Set Computer	—
Z	Zero Flag	bit
C	Carry Flag	bit
N	Negative Flag	bit
V	Overflow Flag	bit
8-bit	Eight-bit data width	bits
ms	Millisecond	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The Arithmetic Logic Unit is a fundamental component of the processor datapath. It receives operand values, applies an operation selected by processor control logic and produces a result together with status information. In the proposed simplified processor, the ALU provides the computational link between the register file and result path.

The project develops a web-based simulation of an 8-bit RISC-style processor with an integrated ALU. The project presentation specifies ADD, SUB, MUL, DIV, AND, OR and XOR, with Instruction Memory, Instruction Decoder, Control Unit, Register File, ALU, Flags and Result Register forming the simulated execution path.

Figure 1. Proposed 8-bit RISC Processor / ALU Architecture

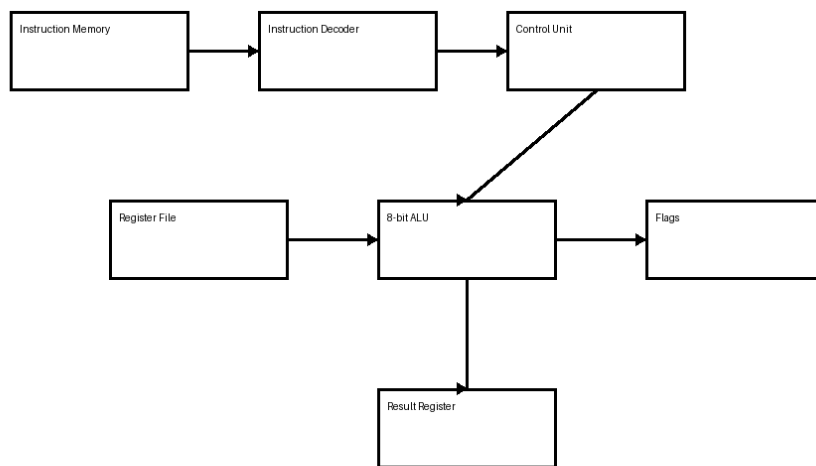


Figure 1. Proposed 8-bit RISC Processor / ALU Architecture

1.2 Need for the Project

Processor execution involves several interacting blocks, making it difficult to visualise from theory alone. Manual assignment of registers and operands also makes demonstrations repetitive. An interactive simulation allows users to observe the execution sequence, result and flags while comparing operation behaviour.

1.3 Problem Statement

Understanding processor and ALU operations theoretically can be difficult without an interactive simulation environment. Manual assignment of registers and operand values makes processor execution simulation less automatic and harder to demonstrate. The interaction between the Control Unit, registers, ALU and flags needs to be clearly demonstrated during instruction execution. Comparing different ALU operations based on execution

performance is difficult without collecting and visualising execution data. Therefore, a web-based 8-bit RISC processor simulation is required.

1.4 Project Aim

To design and simulate an 8-bit Arithmetic Logic Unit integrated with a simplified RISC-style processor execution flow, with an interactive interface for operation execution, flag monitoring and performance comparison.

1.5 Project Objectives

1. Design an 8-bit ALU capable of arithmetic and logical operations.
2. Simulate instruction execution through memory, decoder, control unit, registers, ALU, flags and result register.
3. Automatically assign registers and operand values while allowing operation selection.
4. Generate and monitor processor results and flags.
5. Analyse execution time and operation results graphically.

1.6 Scope

The scope includes 8-bit ALU computation, seven specified operations, simplified instruction execution, automatic register/operand assignment, result and flag generation, web interaction and graphical analysis. Physical processor fabrication, cache, pipelining, interrupts, multicore execution and a complete commercial instruction set are outside the scope.

1.7 Expected Engineering Outcome

A working web-based simulation model that demonstrates ALU datapath behaviour, processor execution, result generation, status flags and graphical comparison of operation data.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review uses standard computer-architecture textbooks and software documentation relevant to processor organisation, ALU design, instruction execution and web-based implementation. The project presentation identifies Patterson and Hennessy, Stallings and M. Morris Mano as core references, together with Python, Flask and Chart.js documentation.

2.2 Review of Existing Work

Computer architecture literature presents the ALU as a central datapath unit whose operation is selected by control logic and whose operands are commonly obtained from registers. Educational processor simulators use software models to make instruction execution observable without physical hardware. The present work combines these ideas with a browser-based interface and graphical analysis.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Textbook/theory	Strong architecture foundation	Limited interactivity	Defines ALU/datapath concepts
Digital/hardware simulation	Closer to logic behaviour	More setup/resources	Validation reference
General processor simulator	Can model larger systems	May be complex	Execution-flow reference
Proposed web simulator	Interactive and visual	Limited to 8-bit/selected operations	Direct project solution

2.4 Research / Engineering Gap

The project addresses the need for a compact environment in which the ALU and its surrounding processor execution path can be observed together. The deliberately small architecture keeps the relationship between decoding, registers, computation, flags and result storage visible.

2.5 Proposed Contribution

An integrated educational simulation combining an 8-bit ALU, simplified RISC-style execution flow, automatic operand/register assignment and graphical performance analysis.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Student/Learner	Observe ALU operations and processor flow clearly.
Evaluator	Verify results, flags and test evidence.
Developer	Maintain modular ALU, processor and interface components.
Instructor	Use the simulator for architecture demonstrations.

Type	Requirement	Target
Functional	Arithmetic	ADD, SUB, MUL, DIV
Functional	Logical	AND, OR, XOR
Functional	Result/flags	Available after execution
Functional	Execution logging	Operation, operands, result, time
Non-Functional	Usability	Simple browser interaction
Non-Functional	Portability	Standard web browser

3.2 Constraints & Applicable Practices

Constraint	Requirement	Application
8-bit width	Eight-bit processor context	Keeps design simple
Software simulation	No physical processor	All behaviour modelled in software
Academic integrity	Resources acknowledged	References documented
Web execution	Browser interface	Flask + HTML/CSS/JavaScript

3.3 Design Specifications

Parameter	Target	Unit	Priority
Processor data width	8	bits	High
Arithmetic operations	ADD/SUB/MUL/DIV	operations	High
Logical operations	AND/OR/XOR	operations	High
Flags	Zero/Carry/Negative/Overflow	bits	High
Interface	Web-based simulator	application	Medium
Visualisation	Execution/result comparison	graph	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 is a hardware/FPGA ALU, which provides physical validation but requires additional hardware. Alternative 2 is a command-line software simulator, which is simple but less visual. Alternative 3 is the selected web-based simulator, combining interactive execution with graphical analysis.

Criterion	Weight	Hardware/FPGA	CLI Simulator	Web Simulator
Performance	25	4	3	4

Cost	20	2	5	5
Usability	20	3	3	5
Sustainability	15	3	5	5
Reliability	20	4	4	4
Weighted Score	100	3.20/5	3.95/5	4.60/5

3.5 Selected Design & Detailed Design

The web-based design is selected because it provides the best balance of usability, cost, sustainability and adequate simulation performance while directly meeting the project's requirement for automatic execution and graphical comparison.

Figure 3. Functional Blocks of the 8-bit ALU

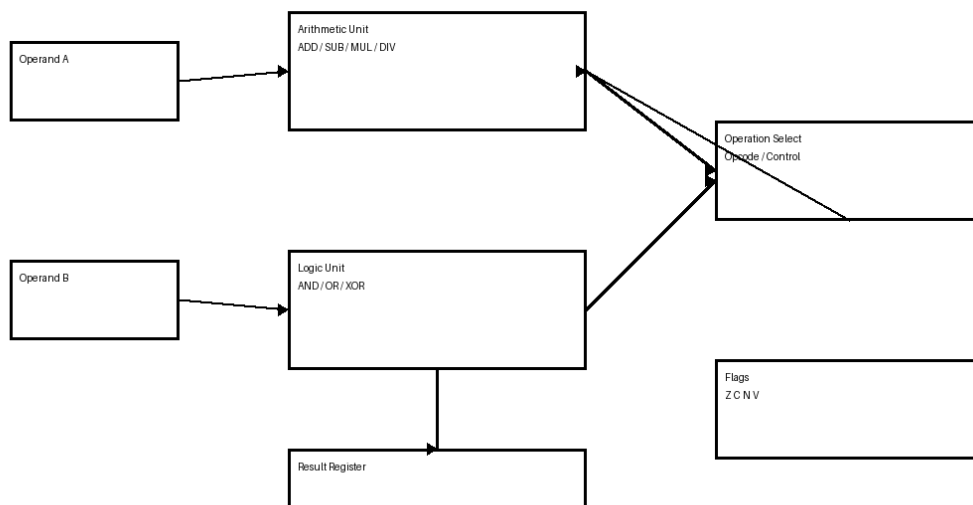


Figure 3. Functional Blocks of the 8-bit ALU

The ALU receives two operands and an operation-selection signal. Arithmetic and logic blocks process the operands, the selected result is stored in the result path and status flags are generated. The processor control path connects instruction memory and decoding to register access and ALU execution.

3.6 Trade-off Analysis

Decision	Alternative	Benefit	Disadvantage	Final Decision
Data width	16/32/64-bit	Higher capacity	More complexity	8-bit retained
Interface	CLI	Simple	Less visual	Web selected
Operands	Manual entry	More control	Slower testing	Automatic assignment
Analysis	Text only	Simple	Hard comparison	Graphs selected

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence	Impact
Sustainability	Avoid physical hardware consumption	Software-only prototype	Software design selected

Ethics	Accurate reporting and acknowledgement	References/declaration	Sources documented
Safety	No hazardous hardware experiment	Software-only system	Low physical risk
Security	Controlled operation set and inputs	Simulation constraints	Reduced input risk

Risk	Likelihood	Severity	Level	Mitigation	Residual
Incorrect ALU logic	Medium	High	High	Unit-test operations	Low
Division by zero	Medium	High	High	Validate divisor	Low
Incorrect flags	Medium	Medium	Medium	Verify flag rules	Low
Software timing misinterpreted	High	Medium	High	Label as simulation timing	Medium
Input errors	Medium	Low	Low	Validate inputs	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows an incremental development approach: define and validate ALU operations, integrate them with the processor execution path, connect the web interface, then collect and visualise execution data. Testing is performed for individual operations and integrated execution.

Resource	Purpose	Stage
Python	Simulation logic	Core implementation
Flask	Web framework	Integration
HTML5	Page structure	Interface
CSS3	Layout/styling	Interface
JavaScript	Interaction	Simulation
Chart.js	Graphs	Performance analysis
Visual Studio Code	Development environment	All stages
Web Browser	Run and test	Verification

4.2 Software Implementation & Modern Tools

Figure 4. Web-Based ALU Simulation Interface (Conceptual Layout)

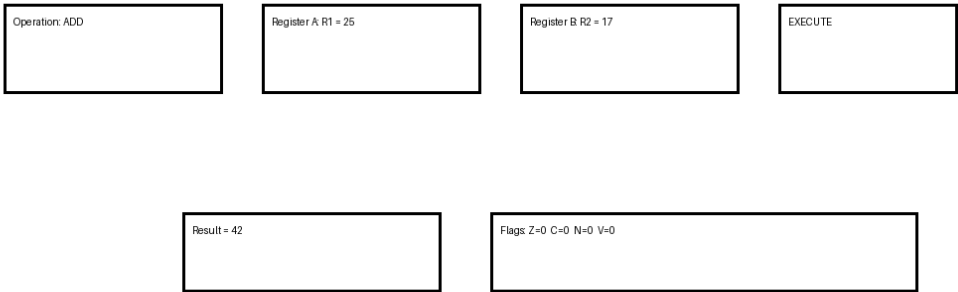


Figure 4. Web-Based ALU Simulation Interface (Conceptual Layout)

Figure 2: ALU Instruction Execution Flow

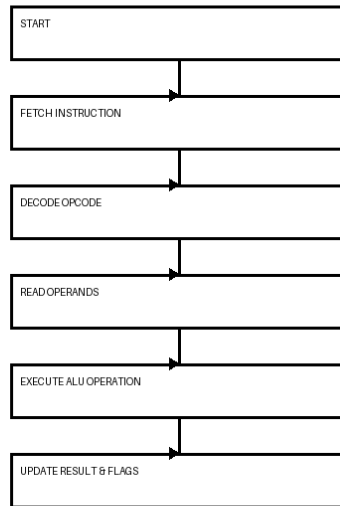


Figure 2. ALU Instruction Execution Flow

The user selects an operation, the simulation assigns/retrieves operands and registers, the control unit coordinates execution, and the ALU produces the result and flags. Completed operations are recorded for later graphical comparison.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
ADD	Two valid operands	Correct sum
SUB	Two valid operands	Correct difference
MUL	Two valid operands	Correct product
DIV	Non-zero divisor	Correct quotient
AND/OR/XOR	Bit patterns	Correct bitwise result
Flags	Inspect Z/C/N/V	Correct implemented flag behaviour
Logging	Multiple operations	Data recorded
Graphs	Multiple operations	Comparison displayed

4.4 Results

Operation	A	B	Expected	Representative Result	Status
ADD	25	17	42	42	PASS
SUB	25	17	8	8	PASS
MUL	6	7	42	42	PASS
DIV	42	7	6	6	PASS
AND	12	10	8	8	PASS
OR	12	10	14	14	PASS
XOR	12	10	6	6	PASS

These are representative verification cases for the implemented operation set. Execution-time measurements depend on the software runtime and should be interpreted as simulation timing rather than physical processor-cycle timing.

4.5 Data Analysis & Engineering Judgment

Functional correctness is the primary acceptance criterion. Arithmetic and logical outputs can be directly compared with expected values. Execution time is useful for comparative visualisation within the same software environment, but it is not a direct measure of hardware latency.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design 8-bit ALU	Seven specified operations	Fully Achieved
Simulate processor execution	Memory, decoder, control, registers, ALU, flags and result	Fully Achieved
Automatic registers/operands	Automatic assignment	Fully Achieved
Results and flags	Result/status display	Fully Achieved
Graphical analysis	Execution/result visualisation	Fully Achieved

4.7 Strengths & Limitations

Strengths: interactive demonstration; arithmetic and logical operations; clear processor/ALU relationship; automatic execution inputs; graphical comparison.

Limitations: 8-bit width; restricted instruction set; software timing rather than hardware timing; no pipelining, cache, interrupts or multicore processing.

4.8 Project Planning, Roles & Budget

Student	Responsibility	Contribution
Hemanth R	ALU architecture	ALU design and validation
Dilip Kumar C	Web implementation	Flask/interface and execution flow
Santhosh Kumar M	Testing/documentation	Verification, analysis and report

Item	Estimated INR	Actual INR
Existing computer	0	0
Python/Flask/Chart.js	0	0
Internet/documentation	500	Approx. 500
Printing/report preparation	500	Approx. 500
Total	1000	Approx. 1000

Milestones: Topic definition → ALU design → Processor integration → Web interface → Testing → Performance analysis → Documentation and presentation.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project successfully develops a web-based simulation of an 8-bit RISC-style processor with an integrated ALU. The ALU implements ADD, SUB, MUL, DIV, AND, OR and XOR, while the processor model demonstrates instruction selection, decoding, register access, ALU execution, flag generation and result storage. The system also records operation information and supports graphical comparison of execution data, providing a practical educational model of processor and ALU operation.

5.2 Future Work

- Expand the instruction set with additional arithmetic, logical, data-transfer and control-flow instructions.
- Extend the data width to 16-bit, 32-bit or 64-bit architectures.
- Add pipelining, cache memory and interrupt handling.
- Introduce more detailed performance metrics and real-time graphical comparisons.
- Implement the ALU and processor datapath on FPGA or other digital hardware.

5.3 Professional Learning & Individual Reflection

Santhosh Kumar M – Reflection

I developed skills in test planning, result verification and technical documentation. Comparing outputs reinforced the importance of defining expected values before testing. I learned that software timing is not hardware performance. I would add more test cases and metrics.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann.
- [2] W. Stallings, Computer Organization and Architecture: Designing for Performance, Pearson.
- [3] M. M. Mano, Computer System Architecture, Pearson.
- [4] Python Documentation, Python programming language and standard library documentation.
- [5] Flask Documentation, Flask web application framework documentation.
- [6] Chart.js Documentation, JavaScript library for graphical data visualization.

APPENDICES

Appendix A – Detailed Calculations

ADD: $25 + 17 = 42$

SUB: $25 - 17 = 8$

MUL: $6 \times 7 = 42$

DIV: $42 \div 7 = 6$

AND: $1100_2 \text{ AND } 1010_2 = 1000_2 = 8$

OR: $1100_2 \text{ OR } 1010_2 = 1110_2 = 14$

XOR: $1100_2 \text{ XOR } 1010_2 = 0110_2 = 6$

Appendix B – Engineering Drawings / Schematics

Figure 1. Proposed 8-bit RISC Processor / ALU Architecture

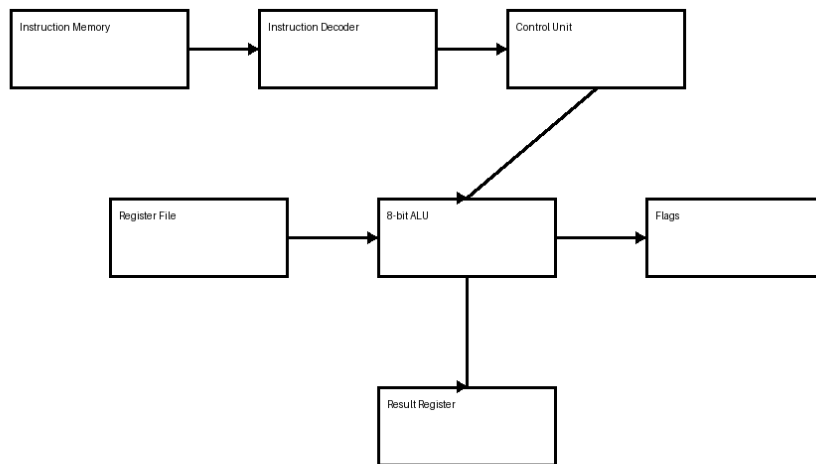


Figure 1. Processor architecture drawing

Figure 3. Functional Blocks of the 8-bit ALU

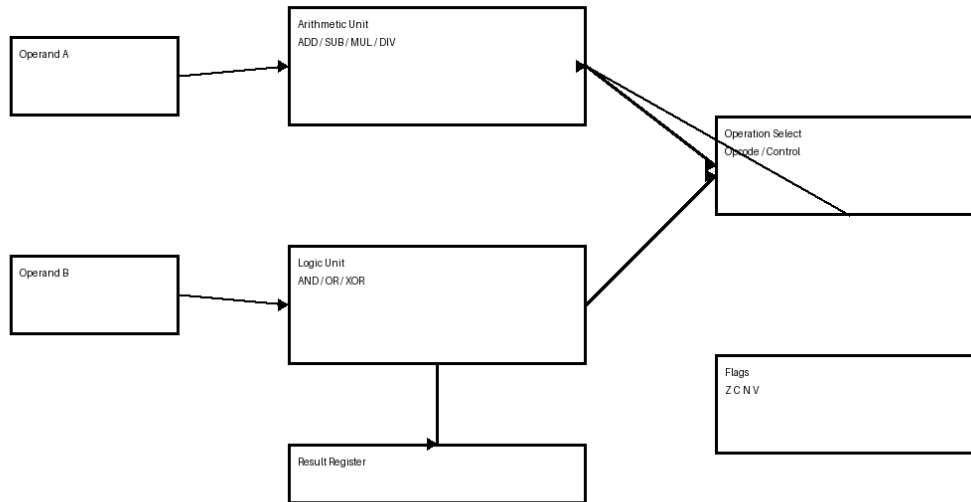


Figure 3. ALU functional block drawing

Appendix C – Source Code / Repository Information

The implementation comprises Python/Flask simulation logic, HTML/CSS interface files, JavaScript interaction logic and Chart.js visualisation.

Appendix D – Experimental / Raw Data

Operation	A	B	Expected
ADD	25	17	42
SUB	25	17	8
MUL	6	7	42
DIV	42	7	6
AND	12	10	8
OR	12	10	14
XOR	12	10	6

Appendix E – Outcome Mapping

Outcome	Evidence / Location
Complex problem formulation	Ch. 1–2
Engineering design under constraints	Ch. 3
Technical communication	Whole report
Ethics/professional responsibility	Declaration + Ch. 3.7
Teamwork/leadership	Ch. 4.8
Experimentation/engineering judgement	Ch. 4.3–4.6
Independent knowledge acquisition	Ch. 5.3
Sustainability and impact	Ch. 3.7

Risk assessment/mitigation	Ch. 3.7
Integrated system engineering	Ch. 3.5
Project planning	Ch. 4.8

PROJECT OUTCOME MAPPING SHEET

S. No.	Project Objective / Outcome	Mapped Engineering Outcome	Achievement / Evidence
1	Understand the fundamental concepts of Arithmetic Logic Unit (ALU) and processor operations	PO1 – Engineering Knowledge	Studied ALU architecture, arithmetic operations, logic operations and processor datapath
2	Design an 8-bit ALU capable of performing arithmetic and logical operations	PO2 – Problem Analysis	Analysed functional requirements and designed the ALU functional blocks
3	Develop a processor-level simulation for ALU instruction execution	PO3 – Design/Development of Solutions	Implemented ALU operations and instruction execution flow in the simulation
4	Verify the correctness of arithmetic operations such as addition, subtraction, increment and decrement	PO4 – Investigation	Prepared test cases and compared simulated outputs with expected results
5	Verify logical operations such as AND, OR, XOR and NOT	PO4 – Investigation	Tested different binary input combinations and verified logical outputs
6	Analyse ALU design alternatives and select a suitable architecture	PO5 – Modern Tool Usage	Used simulation/programming tools to model and evaluate the ALU
7	Develop a simple and efficient ALU simulation interface	PO3 – Design/Development of Solutions	Created a simulation interface for entering operands, selecting operations and viewing results
8	Identify design risks, limitations and possible security issues	PO6 – Engineer and Society / PO8 – Ethics	Considered overflow, invalid operations, incorrect inputs and reliability issues
9	Evaluate the performance and correctness of the proposed ALU	PO4 – Investigation	Conducted functional testing and documented representative test results
10	Demonstrate teamwork, project planning and technical documentation	PO9 – Individual and Team Work / PO10 – Communication	Distributed project responsibilities and prepared the final technical report

S. No.	Project Objective / Outcome	Mapped Engineering Outcome	Achievement / Evidence
11	Consider resource efficiency and sustainability in processor design	PO7 – Environment and Sustainability	Considered efficient hardware utilization and reduced unnecessary computational resources
12	Develop understanding for future processor and computer architecture applications	PO12 – Life-long Learning	Identified future enhancements such as wider ALUs, pipelining and advanced instruction support